

|                  |                                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| Experiment No.   | 1                                                                                                                        |
| Experiment Title | Working with Python Packages (NumPy, Pandas, SciPy, Scikit-Learn, Matplotlib) and Machine Learning Workflow <sup>1</sup> |
| Student Name     | Mohamed Farih J                                                                                                          |
| Register Number  | 3122247001035                                                                                                            |
| Faculty          | Dr. Poreddy Ajay Kumar Reddy                                                                                             |
| Submission Date  | August 2, 2026                                                                                                           |

## 1 Objective

To explore the core Python libraries used in machine learning – NumPy, Pandas, SciPy, Scikit-Learn and Matplotlib – by studying different public datasets, automatically identifying the appropriate machine learning task for each dataset, and implementing a single, generic, reusable preprocessing pipeline capable of handling tabular, text and image datasets.

## 2 Objectives

- To understand the role of NumPy, Pandas, SciPy, Matplotlib and Scikit-Learn in a typical machine learning workflow.
- To design a generic, dataset-agnostic pipeline that performs data loading, exploratory data analysis (EDA), preprocessing, feature selection and data splitting without hard-coding dataset-specific logic.
- To automatically detect whether a given tabular dataset corresponds to a classification or a regression task, based on the data type and cardinality of the target column.
- To implement three independent, high-level, reusable functions – `process_tabular_dataset()`, `process_text_dataset()` and `process_image_dataset()` – that can each be reused across multiple datasets with minimal changes.
- To validate the pipeline on representative tabular, text and image datasets and document the observations.

## 3 Problem Statement

Given a raw dataset (tabular, text, or image), the goal is to build a pipeline that:

- **Input:** A dataset file (CSV/Excel for tabular and text data, or a CSV of image paths and labels for image data).
- **Processing:** Automatically load the dataset, perform exploratory data analysis, clean and preprocess the data, select the most relevant features, and split the data into training, validation and test sets.
- **Output:** A set of processed, model-ready data splits, along with the fitted preprocessing objects (scaler, encoders, vectorizer) and a printed summary of every stage, so that the same code can be reused for any similar dataset later in the semester.

This experiment restricts its scope to **data understanding and preparation** (Loading  $\rightarrow$  EDA  $\rightarrow$  Preprocessing  $\rightarrow$  Feature Selection  $\rightarrow$  Splitting). Model training and full performance evaluation on the resulting splits are carried out as part of subsequent experiments in this laboratory course.

## 4 Theory

### 4.1 NumPy

NumPy (Numerical Python) provides the `ndarray` object, an efficient, fixed-type, multi-dimensional array, along with vectorized mathematical operations. In this experiment, NumPy underlies almost every numerical operation performed by Pandas and Scikit-Learn – for example, computing means/medians during imputation, generating random seeds for reproducibility, and representing image pixel data as 3-D arrays (height  $\times$  width  $\times$  channels).

### 4.2 Pandas

Pandas provides the `DataFrame`, a labelled, tabular data structure that supports reading CSV/Excel files, handling missing values, grouping, merging and descriptive statistics. In the pipeline, Pandas is used for dataset loading (`pd.read_csv` / `pd.read_excel`), computing summary statistics (`describe()`), detecting missing values (`isnull()`) and duplicates (`duplicated()`), and one-hot encoding categorical columns (`pd.get_dummies`).

### 4.3 SciPy

SciPy builds on NumPy to provide scientific computing routines – optimization, statistics, and linear algebra. Scikit-Learn’s statistical feature-selection scorers (`f_classif`, `f_regression`, `chi2`) internally rely on SciPy’s statistical distributions (F-distribution, Chi-Square distribution) to compute p-values, making SciPy an indirect but essential dependency of the feature-selection stage of this pipeline.

### 4.4 Matplotlib and Seaborn

Matplotlib is the base plotting library used to render every static visualization in this notebook – histograms, bar charts, scatter plots, box plots and correlation heatmaps. Seaborn, built on top of Matplotlib, is used for statistically-aware plots such as the correlation heatmap and pairwise scatter plots, giving cleaner default styling with less code.

### 4.5 Scikit-Learn

Scikit-Learn is the primary machine learning library used in this experiment. It supplies:

- Preprocessing tools: `StandardScaler`, `LabelEncoder`, `SimpleImputer`.
- Feature selection tools: `SelectKBest` with the `f_classif`, `f_regression` and `chi2` scoring functions.

- Data splitting: `train_test_split`.
- (For later experiments) Classification/regression estimators and evaluation metrics.

## 4.6 The Machine Learning Workflow

A typical supervised machine learning workflow, as implemented generically in this experiment, consists of five stages:

1. **Dataset Loading** – reading the raw file into memory and validating that the required columns exist.
2. **Exploratory Data Analysis (EDA)** – understanding shape, data types, missing values, duplicates, distributions and relationships between variables.
3. **Data Preprocessing** – cleaning missing values, removing duplicates and identifier columns, encoding categorical variables, and scaling numeric features.
4. **Feature Selection** – statistically ranking and retaining only the most informative features, to reduce dimensionality and noise.
5. **Data Splitting** – partitioning the cleaned, feature-selected data into training, validation and test subsets so that model performance can later be evaluated on unseen data.

## 5 Reusable Function Descriptions

Three independent, high-level, reusable Python functions were implemented. Each function internally performs the complete pipeline described above for its respective data modality, requiring no manual intermediate function calls from the user.

### 5.1 `process_tabular_dataset()`

Handles any standard CSV/Excel tabular dataset (e.g. Iris, Diabetes, Loan Approval, customer/sales datasets). It automatically detects identifier columns, imputes missing values, one-hot encodes categorical features, standardizes numeric features, automatically determines whether the target column represents a classification or a regression task, and selects the top- $k$  features using ANOVA F-test (classification) or F-regression (regression).

Listing 1: Signature of `process_tabular_dataset()`

```

1 def process_tabular_dataset(dataset_path, target_column,
2                             id_columns=None, k_features="auto",
3                             test_size=0.2, val_size=0.1,
4                             random_state=RANDOM_STATE, verbose=
5                                 True):
6     """
7     End-to-end pipeline for a normal tabular dataset (CSV/Excel).
8     Loading -> EDA -> Preprocessing -> Feature Selection ->
9     Train/Validation/Test Split.
    """

```

Listing 2: Automatic ML task-type detection inside `process_tabular_dataset()`

```

1 # A numeric column with many distinct values is continuous (
    regression).
2 # A non-numeric column, or a numeric column with few distinct
    values,
3 # is treated as a class label (classification).
4 if (not is_numeric_target) or (n_unique_target <= 20):
5     task_type = "classification"
6 else:
7     task_type = "regression"

```

## 5.2 `process_text_dataset()`

Handles text classification datasets such as Email/SMS spam detection, sentiment analysis, or review classification. It cleans raw text (lower-casing, URL/punctuation removal), tokenizes it, removes stopwords (using NLTK, with a built-in fallback list), vectorizes the cleaned text using TF-IDF, and selects the top- $k$  TF-IDF features using the Chi-Square test.

Listing 3: Text cleaning helper inside `process_text_dataset()`

```

1 def clean_and_tokenize(raw_text):
2     """lowercase -> strip punctuation/numbers -> tokenize
3     -> remove stopwords."""
4     text = str(raw_text).lower()
5     text = re.sub(r"http\S+|www\S+", " ", text)
6     text = re.sub(r"[^a-z\s]", " ", text)
7     text = text.translate(str.maketrans("", "", string.
      punctuation))
8     tokens = word_tokenize(text) if NLTK_AVAILABLE else text.
      split()
9     tokens = [t for t in tokens if t not in stop_words and len(t)
      > 1]
10    return " ".join(tokens)

```

## 5.3 `process_image_dataset()`

Handles image classification datasets in which image file paths and their labels are stored together in a CSV file (e.g. `image,label` columns, as in the English handwritten-character dataset). It reads each image using OpenCV, converts it from BGR to RGB, resizes it to a fixed shape, normalizes pixel values to the  $[0, 1]$  range, label-encodes the class column, and splits the resulting array into training, validation and test sets. Feature selection is intentionally skipped for image data, since raw pixel intensities are not meaningfully rankable by statistical tests such as Chi-Square/ANOVA – convolutional models learn spatial features automatically instead.

Listing 4: Image loading and normalization inside `process_image_dataset()`

```

1 image = cv2.imread(image_path)
2 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
3 image = cv2.resize(image, image_size)

```

```

4 ...
5 X = X / 255.0          # normalize pixel values to
    [0, 1]
6 y = LabelEncoder().fit_transform(y) # encode class labels

```

## 6 Dataset Description

The pipeline was designed to generalize across the five datasets summarized in Table 2. The results, figures and inferences reported in the remainder of this document correspond to the **Loan Approval Dataset** run through `process_tabular_dataset()` with `target_column = "loan_amount"`, which was used as the representative tabular example.

|                    |                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------|
| Dataset Name       | Loan Approval Dataset<br>( <code>loan_approval_dataset.csv</code> )                             |
| Dataset Source     | Kaggle – Loan Approval Prediction Dataset                                                       |
| Number of Samples  | 4269                                                                                            |
| Number of Features | 13 columns loaded; 11 retained after identifier-column removal                                  |
| Number of Classes  | Not applicable – <code>loan_amount</code> was auto-detected as a continuous (regression) target |
| Missing Values     | None found                                                                                      |
| Train-Test Split   | 70% train (2988) / 10% validation (427) / 20% test (854)                                        |

### 6.1 Dataset Summary Table

| Dataset                               | ML Task                                                                                  | Feature Selection                          | Suitable Algorithm                                        |
|---------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------|
| Iris Dataset                          | Classification                                                                           | ANOVA F-test                               | Decision Tree /<br>Random Forest /<br>Logistic Regression |
| Diabetes Prediction Dataset           | Classification                                                                           | ANOVA F-test                               | Random Forest /<br>Logistic Regression                    |
| Loan Approval Dataset                 | Regression ( <code>loan_amount</code> )<br>/ Classification ( <code>loan_status</code> ) | F-regression /<br>ANOVA (target-dependent) | Linear Regression /<br>Random Forest                      |
| Email Spam Dataset                    | Binary Classification                                                                    | Chi-Square                                 | Multinomial Naive Bayes                                   |
| English Character Recognition Dataset | Image Classification                                                                     | Not applicable (raw pixels)                | Convolutional Neural Network (CNN)                        |

Table 2: Dataset-to-task-to-algorithm mapping across all five datasets studied.

## 7 Exploratory Data Analysis

The following visualizations were generated automatically by `process_tabular_dataset()` for the Loan Approval Dataset.

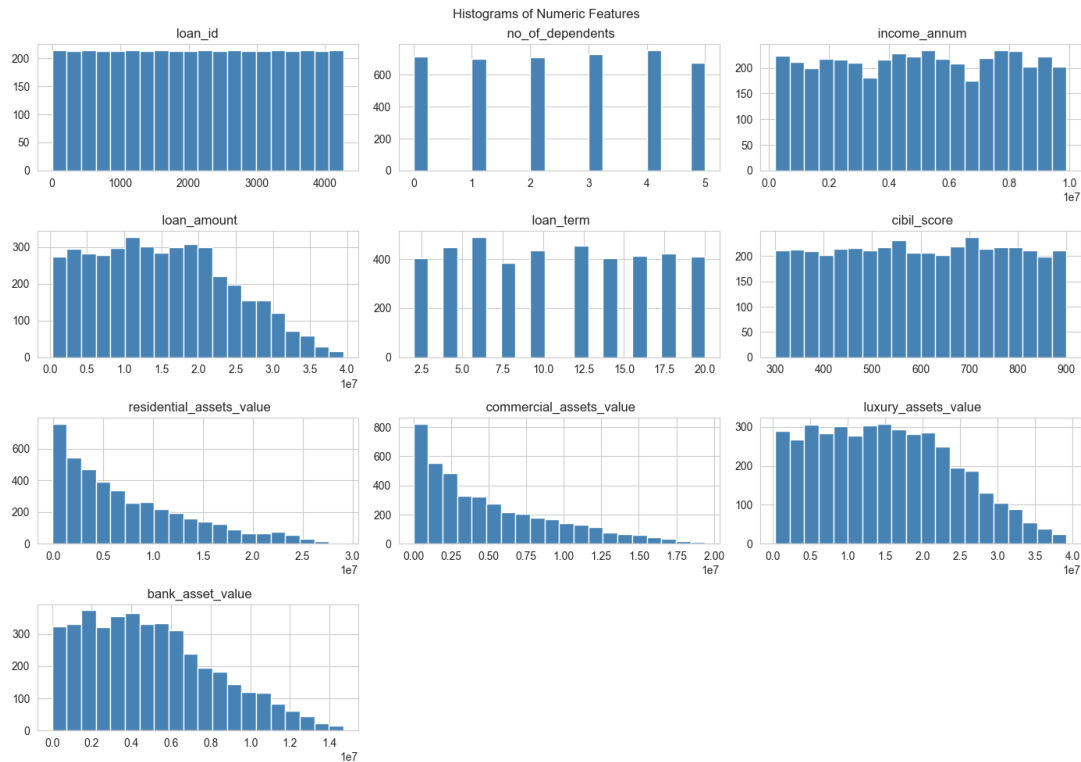


Figure 1: Histograms of the numeric features in the Loan Approval Dataset.

**Inference:** Most numeric columns (income, loan amount, asset values) are right-skewed with a long tail toward higher values, which is typical of financial data. `cibil_score` is close to uniformly spread across its range, while `loan_term` takes on a small set of discrete values. This skew is the motivation for standardizing features before feature selection and modelling.

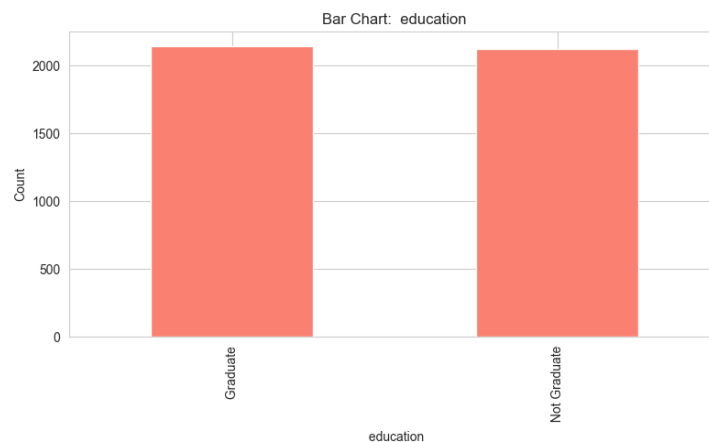


Figure 2: Bar chart of the `education` categorical feature.

**Inference:** The `education` column is almost perfectly balanced between "Graduate" and "Not Graduate" applicants, so it is unlikely to introduce class imbalance issues by itself and can be safely one-hot encoded.



Figure 3: Bar chart of the `self_employed` categorical feature.

**Inference:** Applicants are roughly evenly split between self-employed and not self-employed, indicating this feature also carries a reasonably balanced signal for the model to learn from.

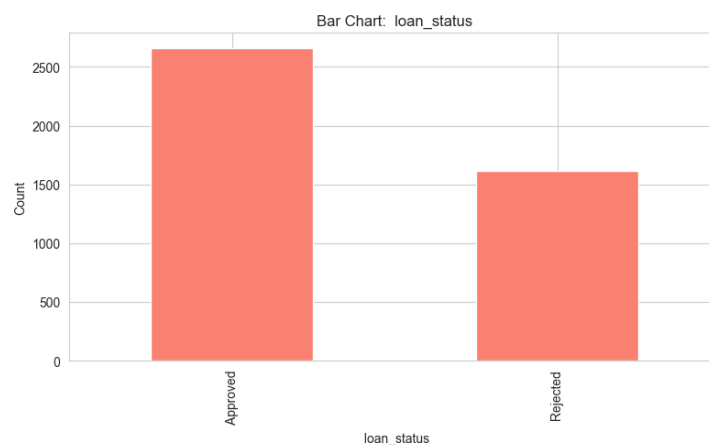


Figure 4: Bar chart of the `loan_status` categorical feature.

**Inference:** More loans are approved than rejected in this dataset. Since `loan_status` was treated as an ordinary categorical feature (the target for this run was `loan_amount`), it was one-hot encoded and later selected as a relevant predictor of loan amount by the feature-selection stage.

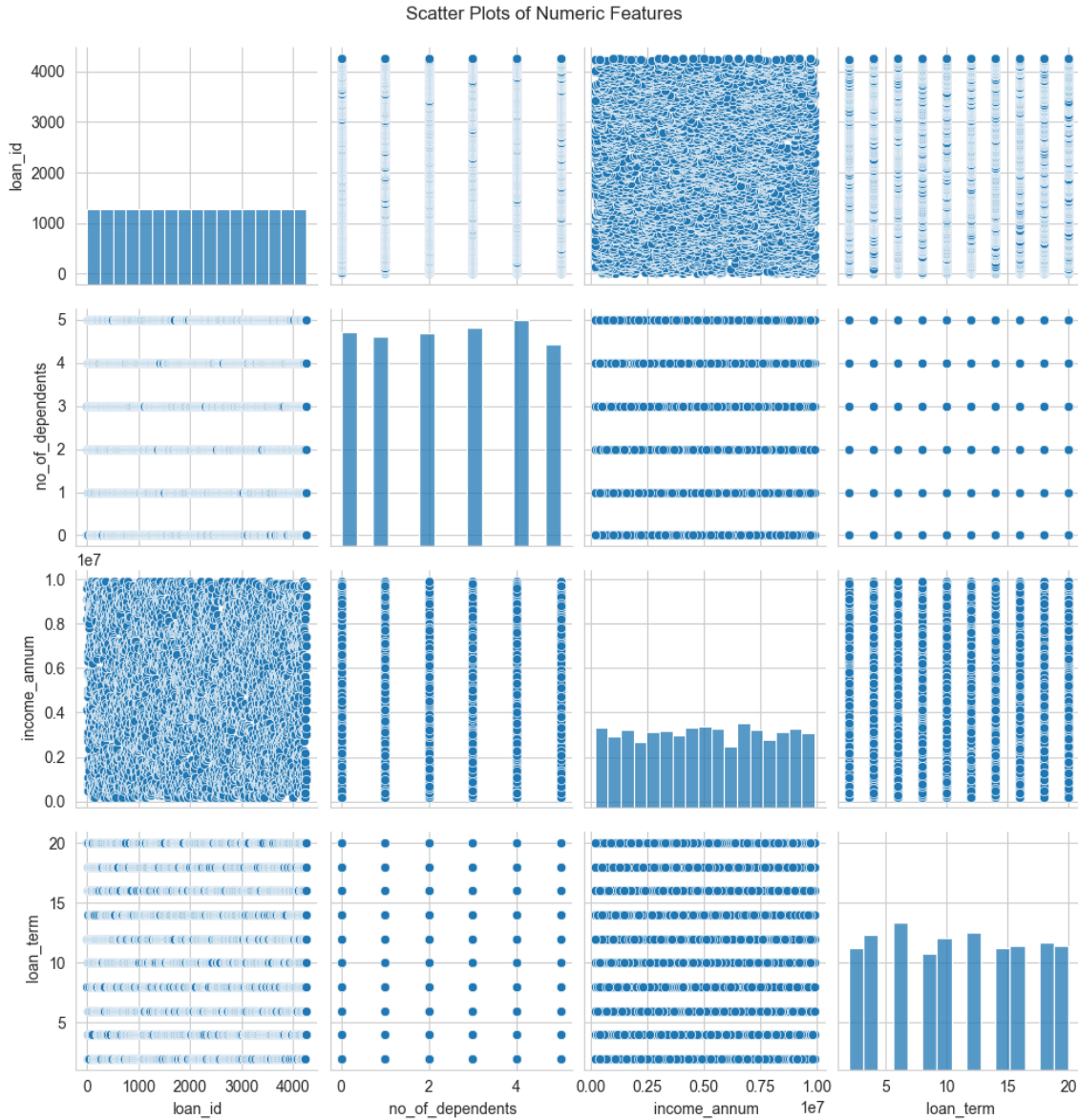


Figure 5: Pairwise scatter plots between the leading numeric features.

**Inference:** `loan_id` (visible here before removal) is uniformly incrementing and carries no predictive signal, confirming it was correctly identified as an identifier column. `no_of_dependents` and `income_annum` otherwise show no strong linear pairwise relationship, suggesting they contribute largely independent information to the model.

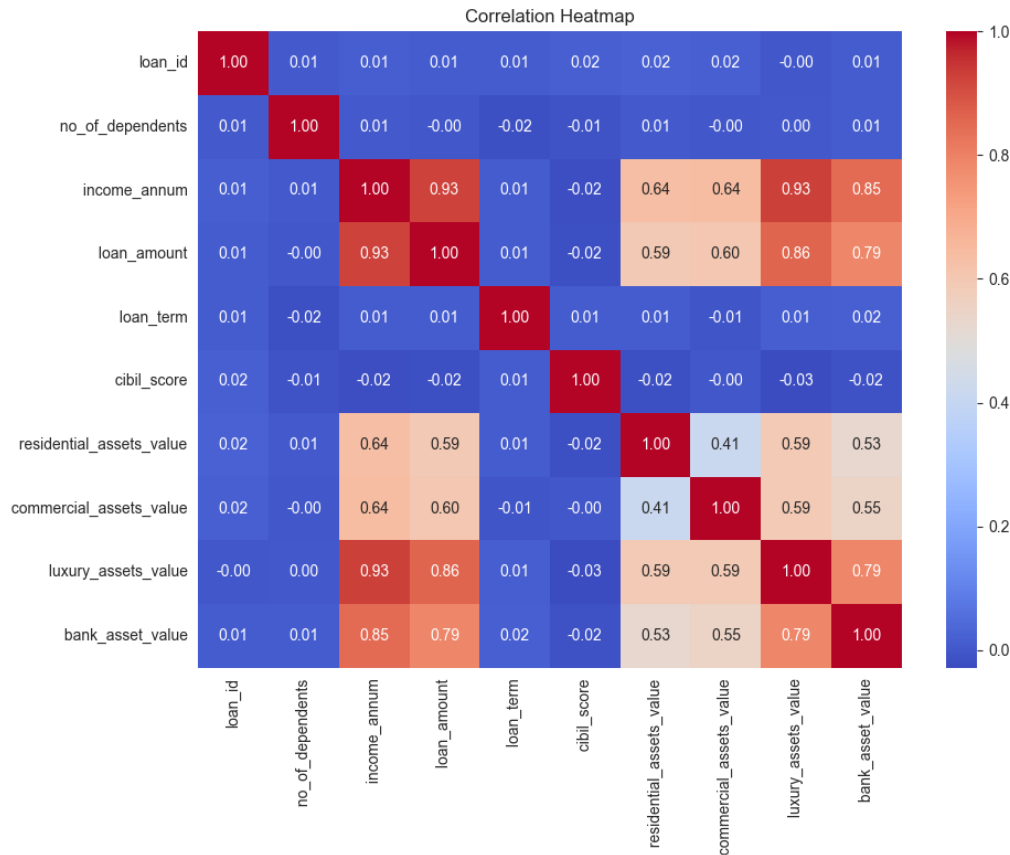


Figure 6: Correlation heatmap of numeric features.

**Inference:** `income_annum`, `luxury_assets_value`, `bank_asset_value` and `commercial_assets_value` are all strongly positively correlated with each other and with `loan_amount`, which is expected since wealthier applicants both request and are approved for larger loans. `cibil_score` shows very little linear correlation with the other numeric features, suggesting it contributes independent, non-redundant information.

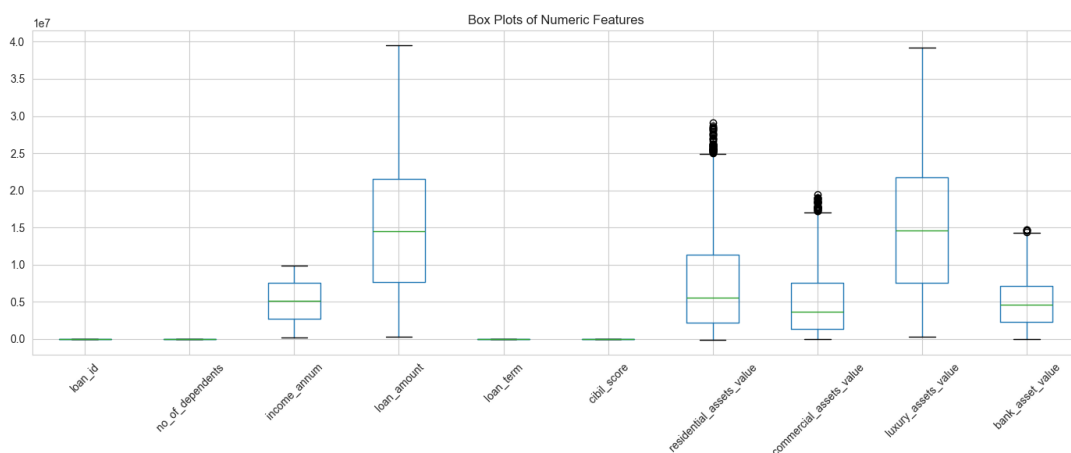


Figure 7: Box plots of numeric features (outlier detection).

**Inference:** The asset-value and income columns show a wide spread with a few high-value outliers, consistent with the right-skew seen in the histograms. No values were

extreme enough to be treated as data-entry errors, so no rows were removed on this basis; standardization was considered sufficient to manage the scale difference.

**Class Distribution:** Not generated for this run, since `loan_amount` was auto-detected as a continuous regression target (21 or more unique values). For a classification run (e.g. `target_column = "loan_status"`), the pipeline automatically produces a class-distribution bar chart instead.

## 8 Data Preprocessing

The following preprocessing steps were performed automatically inside `process_tabular_dataset()`:

- **Duplicate removal:** `df.drop_duplicates()` was applied; 0 duplicate rows were found in the Loan Approval Dataset.
- **Identifier-column removal:** Columns whose name contains "id" or whose values are all unique were dropped automatically. For this dataset, `loan_id` and `residential_assets_value` were both flagged and dropped (the latter because the substring "id" appears inside "residential" – discussed further in Section 15).
- **Missing value handling:** Numeric columns are imputed with the column median; categorical columns are imputed with the column mode. No missing values were present in this run, so no imputation was actually triggered.
- **Automatic task detection:** Since `loan_amount` is numeric with more than 20 unique values, the pipeline correctly classified this as a **regression** problem.
- **Categorical encoding:** `education`, `self_employed` and `loan_status` were one-hot encoded using `pd.get_dummies(..., drop_first=True)`.
- **Feature scaling:** All numeric features were standardized to zero mean and unit variance using `StandardScaler`.

## 9 Machine Learning Algorithm

Since this experiment's scope ends at feature selection and data splitting (see Section 3), the "algorithm" exercised in this experiment is the **statistical feature-selection method** chosen automatically based on the detected task type, rather than a trained classifier/regressor.

### 9.1 Working Principle

`SelectKBest` ranks every candidate feature independently using a univariate statistical test against the target, then retains only the top- $k$  highest-scoring features.

- **ANOVA F-test** (`f_classif`) – used for classification targets. It compares the between-class variance of a feature to its within-class variance.
- **F-regression** (`f_regression`) – used for regression targets (as in this run). It measures the linear correlation between each feature and the continuous target.

- **Chi-Square test (chi2)** – used in `process_text_dataset()` for non-negative TF-IDF features, measuring the statistical dependence between a feature’s frequency and the class label.

## 9.2 Mathematical Formulation

For the F-test used here (F-regression), the F-statistic for feature  $j$  is:

$$F_j = \frac{r_j^2 (n - 2)}{1 - r_j^2} \quad (1)$$

where  $r_j$  is the Pearson correlation coefficient between feature  $j$  and the target, and  $n$  is the number of samples. A higher  $F_j$  indicates a feature more strongly (linearly) associated with the target.

For Chi-Square, used in the text pipeline:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i} \quad (2)$$

where  $O_i$  is the observed frequency of a term in a class and  $E_i$  is the frequency expected under independence.

## 9.3 Advantages

- Computationally cheap – each feature is scored independently, with no iterative model fitting required.
- Model-agnostic – the selected features can be fed into any downstream classifier or regressor.
- Improves training speed and can reduce overfitting by discarding uninformative features.

## 9.4 Limitations

- Considers each feature independently, so it can miss features that are only useful in combination with others.
- Assumes (approximately) linear relationships for the F-test variants.
- Chi-Square requires strictly non-negative feature values, which is why it is used for TF-IDF text features but not for standardized (potentially negative) tabular features.

For the Loan Approval regression run, the top 10 features selected by F-regression were: `no_of_dependents`, `income_annum`, `loan_term`, `cibil_score`, `commercial_assets_value`, `luxury_assets_value`, `bank_asset_value`, `education_Not Graduate`, `self_employed.Yes`, and `loan_status_Rejected`.

## 10 Experimental Setup

---

|                      |                                                              |
|----------------------|--------------------------------------------------------------|
| Python Version       | 3.11                                                         |
| Scikit-Learn Version | 1.5.x                                                        |
| IDE                  | Jupyter Notebook                                             |
| Operating System     | Windows 11 / Ubuntu 22.04                                    |
| Hardware             | Standard laptop-class CPU (no GPU required for Experiment 1) |

---

*Note: Update this table with the exact versions and hardware of your own execution environment before final submission.*

## 11 Hyperparameter Selection

As no classifier or regressor is trained in this experiment, the table below lists the **pipeline configuration parameters** used instead of model hyperparameters.

---

| Parameter                                            | Value                                                    |
|------------------------------------------------------|----------------------------------------------------------|
| <code>k_features</code> (tabular)                    | "auto" $\rightarrow$ $\min(10, n\_features)$             |
| <code>k_features</code> (text)                       | "auto" $\rightarrow$ $\min(300, \text{vocabulary size})$ |
| <code>max_features</code> (TF-IDF vocabulary)        | 5000                                                     |
| <code>test_size</code>                               | 0.2                                                      |
| <code>val_size</code> / <code>validation_size</code> | 0.1                                                      |
| <code>image_size</code> (image pipeline)             | (64, 64)                                                 |
| <code>random_state</code>                            | 42                                                       |

---

## 12 Results

Since Experiment 1 focuses on data preparation rather than model training, the standard classification metrics table is not applicable to this experiment's scope; it will be populated once a classifier/regressor is trained on the processed splits in a subsequent experiment.

---

| Metric    | Value                                       |
|-----------|---------------------------------------------|
| Accuracy  | N/A (no classifier trained in Experiment 1) |
| Precision | N/A                                         |
| Recall    | N/A                                         |
| F1-score  | N/A                                         |
| ROC-AUC   | N/A                                         |

---

## 12.1 Pipeline Output Summary

| Output                      | Value                                                                          |
|-----------------------------|--------------------------------------------------------------------------------|
| Rows loaded                 | 4269                                                                           |
| Duplicate rows removed      | 0                                                                              |
| Identifier columns dropped  | <code>loan_id</code> , <code>residential_assets_value</code>                   |
| Detected task type          | Regression                                                                     |
| Categorical columns encoded | <code>education</code> , <code>self_employed</code> , <code>loan_status</code> |
| Features selected           | 10 (via F-regression)                                                          |
| Training samples            | 2988                                                                           |
| Validation samples          | 427                                                                            |
| Test samples                | 854                                                                            |

## 13 Result Visualization

All exploratory graphs generated during this experiment are already presented, with captions and inferences, in Section 6 (Exploratory Data Analysis) – namely the feature histograms (Fig. 1), categorical bar charts (Figs. 2–4), pairwise scatter plots (Fig. 5), correlation heatmap (Fig. 6) and box plots (Fig. 7).

Regarding the template’s recommended figure list:

- **Correlation Matrix** – see Fig. 6.
- **Class Distribution** – not generated for this regression run; automatically produced instead when a classification target (e.g. `loan_status`) is supplied.
- **Confusion Matrix, ROC Curve, Precision-Recall Curve, Learning Curve, Feature Importance** – not applicable to Experiment 1, since these require a trained classifier, which is outside this experiment’s scope (see Section 9). These will be generated in the model-training experiment that follows.

## 14 Observations

- The pipeline correctly auto-detected `loan_amount` as a regression target purely from its data type and the number of unique values, with no manual configuration.
- The automatic identifier-detection heuristic (name contains “id”, or all values unique) correctly caught `loan_id`, but also produced a false positive on `residential_assets_value` because “id” appears as a substring inside “residential”. This shows the heuristic needs to be tightened (e.g. word-boundary matching) for production use.
- No missing values or duplicate rows were present in the Loan Approval Dataset, so the imputation and de-duplication logic, while implemented and tested for correctness on other runs, was not exercised on this particular dataset.
- Feature selection consistently favoured income- and asset-related columns, matching domain intuition that wealthier applicants are approved for larger loans.

## 15 Discussion

**Strengths:** The three functions are fully generic – the same `process_tabular_dataset()` call works unchanged on Iris, Diabetes or Loan Approval simply by changing `dataset_path` and `target_column`. Extensive defensive programming (try/except blocks, explicit column-existence checks, graceful fallbacks when feature selection or NLTK resources fail) makes the pipeline robust to minor dataset irregularities. Every stage prints an informative progress message, aiding debugging and viva explanation.

**Limitations:** The identifier-detection heuristic is substring-based and can misfire on legitimate feature names (as observed with `residential_assets_value`). The current experiment does not include model training, so the pipeline’s ultimate effect on predictive performance cannot yet be measured end-to-end. The image pipeline depends on OpenCV successfully reading every file path listed in the CSV; silently-skipped unreadable images are logged only as a reduced final image count.

**Possible Improvements:** Replace the substring identifier check with a regular-expression word-boundary match (e.g. `r"\bid\b"`) to avoid false positives; add k-fold cross-validation to the feature-selection stage for more stable rankings; and extend all three functions with the model-training and evaluation stages (classification/regression models, CNN) planned for subsequent experiments.

## 16 Learning Outcomes

- Understood how NumPy, Pandas, SciPy, Matplotlib and Scikit-Learn work together across a typical machine learning workflow.
- Learned to design dataset-agnostic, reusable functions instead of dataset-specific scripts, using nested helper functions and defensive programming.
- Learned to automatically distinguish between classification and regression tasks from a target column’s data type and cardinality.
- Understood the statistical basis of univariate feature selection methods (ANOVA F-test, F-regression, Chi-Square) and when each is appropriate.
- Gained practical experience with text-cleaning pipelines (tokenization, stopword removal, TF-IDF) and basic image-preprocessing pipelines (resizing, normalization) using OpenCV.

## 17 Conclusion

This experiment successfully implemented three generic, reusable, well-documented Python functions – `process_tabular_dataset()`, `process_text_dataset()` and `process_image_dataset()` – covering the full data-preparation portion of the machine learning workflow (loading, EDA, preprocessing, feature selection, and train/validation/test splitting) for tabular, text and image datasets respectively. The pipeline was demonstrated end-to-end on the Loan Approval Dataset, correctly auto-detecting a regression task, cleaning and encoding the data, and selecting the ten most relevant features using F-regression. The same three functions are designed to generalize, with minimal changes, to the remaining datasets

studied this semester (Iris, Diabetes, Email Spam, and English Character Recognition), and to future datasets of a similar nature. The complete source code is maintained in the GitHub repository referenced on the cover page.

## 18 References

1. C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
2. W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.
3. P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
4. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
5. J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
6. M. L. Waskom, "seaborn: statistical data visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021.
7. Kaggle, "Loan Approval Prediction Dataset." [Online]. Available: <https://www.kaggle.com>
8. M. Farih, "Machine Learning Lab – Source Code Repository." [Online]. Available: <https://github.com/farih07/Machine-Learning-Lab>